Domain ISA specification

1. Overview & Motivation

Overview & Motivation This ISA is designed for a project that uses cough sounds to help detect TB. In rural areas like Lesotho, many people live far from clinics and can't afford expensive health checks. But almost everyone has a phone with a microphone. If we can analyze cough sounds quickly and cheaply, people could get early warnings and seek help soon.

Our ISA is meant to run the signal processing part of this idea. It focuses on things like:

- Turning cough sound into numbers the system can understand.
- Comparing the features with stored TB-positive cough samples.
- Making a simple decision like "possible TB" or "Not TB".

We want the ISA to be simple, use low power, and work well on mobile or embedded devices.

Main design goals:

- Fast and efficient for audio signal processing.
- Low cost and easy to implement.
- Lightweight instruction set that's easy to decode.
- Supports common operations like array handing math, and branching.

This workload matches a RISC style design because RISC has fewer, simpler instructions that can run quickly.

2. Architectural Design Choices

Feature	Decision	Why
Instruction Philosophy	RISC	It's simpler to design and easy to extend later
Registers	16 General-purpose registers(32-bit) and status register	Enough to handle comparisons without wasting space.
Data Types	8-bit, 16-bit, 32-bit	Matches common audio data formats.
Addressing	Immediate, register indirect, base and offset	Enough for arrays and memory access.
Memory Model	Little endian, word-aligned	Easier to implement in hardware
Instruction Format	Fixed 32-bit	Simple do decode and pipeline. R type, I type and B-type.

3. Instruction Set Summary

Category	Mnemonic(s)	Operands	Description
Arithmetic	ADD,SUB,MUL,DIV	rd, rs1, rs2	Perform basic arithmetic operations.
Logical	AND,OR,XOR,NOT	rd, rs1, rs2	Perform bitwise logical operations
Data Movement	LD,ST	rd, [rs1+imm]	Load data from or store data to memory
Immediate	ADDI,MOVI	rd, imm	Add or move immediate value
Branching	BEQ,BNE,BLT,BGT,JMP	rs1, rs2, label	Control program flow based on conditions.
Signal Ops	SQR,ABS,CMPF	rd, rs1, rs2	Square values, take absolute difference, or compare features.
Output	OUT	rs1	Display or send output results

4. Instruction Encoding Summary

R-Format

Funct7	rs2	rs1	Funct3	rd	opcode
7 bits	5 bits	5 bits	3 bits	5 bits	7 bits

I-Format

Imm[11:0]	rs1	Funct3	rd	opcode
12 bits	5 bits	3 bits	5 bits	7 bits

B-Format

Imm[12]	Imm[10:5]	rs2	rs1	Funct3	Imm[4:1]	Imm[11]	opcode
1 bit	6 bits	5 bits	5 bits	3 bits	4 bits	1 bit	7 bits

5. Design Rationale & Trade-offs

- **Simplicity vs Capability:**

  We chose to use a basic feature extraction method that measures the energy of each signal sample (by squaring the values). This approach keeps the system small, easy to understand, and efficient to run on limited hardware. However, it also means the system has limited capability, it does not capture detailed frequency information like an FFT or MFCC would. The trade-off is that while the program is simpler and faster, it cannot analyze coughs as accurately as more advanced methods.

- **Code Density vs Performance:**

  We used fixed 32-bit instructions, which makes the program code slightly larger in memory. However, this choice simplifies instruction decoding, leading to faster and more consistent execution. The trade-off is between compact code size and higher performance, we prioritized performance and simplicity over code density.

- **Hardware Impact:**

  The system requires a simple datapath that includes an Arithmetic Logic Unit (ALU) capable of performing fast multiplication and addition operations. Since the feature extraction and comparison steps rely on these arithmetic functions, efficient ALU performance is important. Additionally, the system needs quick memory access to store and retrieve cough signal data and reference features smoothly during processing.

- **Extensibility:**

  The system is designed in a way that allows future improvements. More advanced AI-oriented instructions, such as FFT or MFCC extensions, can be added later to enhance its ability to process and analyze sound-based data. This makes the processor adaptable for more complex audio and health-related AI tasks in the future without requiring a complete redesign.